

Technology Stack & Engineering Standards — Part 1: Technology Philosophy & Stack Selection

The technology stack of AROH is selected with the understanding that the platform is expected to evolve over many years and support a growing ecosystem of interconnected products rather than a single application. Every technology decision is evaluated against long-term maintainability, ecosystem compatibility, developer productivity, scalability, community support, and architectural flexibility. The objective is not to use the newest technologies for their own sake, nor to optimize prematurely for enterprise-scale traffic, but to adopt mature tools that allow the platform to grow incrementally without forcing disruptive migrations. Technologies should accelerate development while remaining replaceable through clearly defined abstractions. No framework, cloud provider, or third-party service should become inseparable from the architecture.

The frontend foundation is built around React and Next.js because they provide an excellent balance between developer experience, performance, ecosystem maturity, and future scalability. React's component-based architecture aligns naturally with the AROH Design System, enabling reusable interfaces across products. Next.js extends React with routing, rendering strategies, asset optimization, server capabilities, metadata handling, and deployment optimizations, making it an appropriate foundation for both the platform and future products. The use of the App Router encourages modular application design while allowing server-side capabilities to coexist with client-side interactivity. Although the current implementation primarily serves a web application, the business logic is intentionally separated from the UI so that future mobile and desktop applications can consume the same platform capabilities without sharing presentation code.

TypeScript is adopted as the primary programming language because architectural consistency depends heavily on strong typing and explicit contracts. As the ecosystem expands, hundreds of entities, APIs, services, events, and shared models will interact across products. Static typing significantly reduces ambiguity, improves refactoring safety, strengthens IDE support, enables more capable AI-assisted development, and encourages disciplined engineering practices. Strict TypeScript configuration is treated as a quality standard rather than an optional enhancement. The codebase should prefer explicit types over implicit behavior, ensuring that architectural intent remains visible throughout implementation.

The repository follows a monorepo strategy because every product belongs to the same ecosystem and shares common platform capabilities. Rather than distributing infrastructure across multiple repositories, the monorepo centralizes shared packages, documentation, tooling, configuration, and development workflows while preserving clear ownership boundaries between products and platform modules. This approach minimizes dependency duplication, simplifies coordinated refactoring, enables consistent engineering standards, and allows reusable packages such as the AROH Design System (ADS) and AROH SDK (ASDK) to evolve alongside consuming applications. The monorepo is not intended to couple products together but to provide a single engineering workspace where shared capabilities can mature consistently.

The AROH Design System represents the visual foundation of the ecosystem. It should contain reusable UI components, design tokens, spacing rules, typography scales, color palettes, iconography, responsive utilities, accessibility helpers, and motion primitives. Every product should inherit the same visual language,

ensuring that users perceive the ecosystem as one cohesive platform. Product-specific interfaces may introduce specialized components where necessary, but foundational UI elements should always originate from the shared design system. This approach reduces design inconsistency, improves accessibility compliance, simplifies maintenance, and enables visual improvements to propagate throughout the ecosystem with minimal effort.

The AROH SDK functions as the engineering bridge between platform services and consuming products. Rather than exposing raw infrastructure implementations, the SDK should provide strongly typed models, validation schemas, shared utilities, API clients, authentication helpers, wallet helpers, configuration access, permission evaluation, and reusable abstractions. Products interact with platform capabilities through the SDK instead of directly integrating with infrastructure providers. This separation allows infrastructure implementations to evolve independently while preserving stable interfaces for consuming applications. As additional platform services emerge, the SDK becomes the primary entry point through which products interact with the ecosystem.

State management is treated as an implementation concern rather than an architectural dependency. The current use of Zustand is appropriate because it provides a lightweight, predictable mechanism for managing application state without imposing unnecessary complexity. However, business rules should never become tightly coupled to the chosen state library. State containers are responsible only for coordinating user interface behavior, while authoritative business operations remain within platform services. This distinction ensures that future migrations between state management solutions would require minimal architectural change.

Data persistence should remain abstracted behind platform services. Although Firebase currently provides authentication and database capabilities, the architecture intentionally avoids allowing products to depend directly upon Firebase-specific implementations. The platform should define contracts that describe required behavior rather than implementation details. Whether future persistence is provided by Firebase, Supabase, PostgreSQL, MongoDB, or another technology should remain an infrastructure decision rather than an application concern. This abstraction protects the ecosystem from vendor lock-in and simplifies future evolution.

Tailwind CSS serves as the utility-first styling framework because it promotes consistency, rapid development, and close alignment between design tokens and implementation. Rather than scattering custom CSS throughout products, styling should be derived from shared design tokens and utility classes defined by the design system. Custom styles should exist only where reusable abstractions cannot reasonably satisfy design requirements. Maintaining a disciplined styling approach reduces visual drift, simplifies maintenance, and strengthens consistency across products.

Animations are considered part of the platform rather than isolated implementation details. Motion should communicate hierarchy, relationships, continuity, and user feedback without becoming distracting. A shared motion system ensures that every interaction behaves consistently regardless of which product the user is currently using. Timing curves, easing functions, transition durations, loading states, page transitions, hover behaviors, and accessibility-aware reduced-motion alternatives should all originate from centralized motion tokens rather than individual implementations.

Testing, documentation, linting, formatting, dependency management, continuous integration, deployment, and monitoring are treated as first-class engineering concerns rather than supplementary

tooling. Every technology introduced into the ecosystem should improve long-term maintainability and reduce engineering overhead rather than merely solving an immediate implementation problem. The technology stack should remain intentionally conservative, favoring stability, clarity, and architectural integrity over unnecessary experimentation. New technologies should be adopted only when they demonstrably strengthen the platform's ability to evolve while preserving the principles upon which AROH is being built.